Algorithm 1: Overall Agentic Graph Retrieval Framework

Input: User Query Q, Knowledge Graph KG, Vector Database VDB
Output: Grounded Answer A, Citations C, Validation Report R

Receive user query Q;
Invoke **Query Classification Agent**[A1];
Invoke **Query Rewriting Agent**[A2];
Invoke **Query Decomposition Agent**[A3];
foreach *Subquery* q_i **do**
    Invoke **Vector Retrieval Agent**[A4];
    Invoke **Graph Traversal Agent**[A5];
    Invoke **Community Retrieval Agent**[A6];
    Invoke **Evidence Aggregation Agent**[A7];
Invoke **Context Construction Agent**[A8];
Invoke **Answer Generation Agent**[A9];
Invoke **Citation Mapping Agent**[A10];
Invoke **Faithfulness Validation Agent**[A11];
if *Validation Successful* **then**
    Return grounded answer with citations;
else
    Invoke **Self-Critique Agent**[A12];
    Invoke **Feedback Agent**[A13];
    Invoke **Retry Controller Agent**[A14];
    Repeat retrieval until maximum retry limit is reached;
Invoke **Cache Manager Agent**[A15];
Generate validation report R;
return (A, C, R)

Algorithm 2: Query Classification Agent

Input: User Query Q
Output: Query Type T

Receive user query Q;
Analyze query intent and semantic features;
if *Query requests information about a specific entity* **then**
    Classify as **Local Query**;
else
    if *Query requires thematic or community-level reasoning* **then**
        Classify as **Global Query**;
    else if *Query requires both local and global knowledge* **then**
        Classify as **Hybrid Query**;
    else
        Classify as **Simple Lookup Query**;
Forward query and query type to the **Query Rewriting Agent**[A2];
return T

Algorithm 3: Query Rewriting Agent

Input: User Query Q, Query Type T
Output: Rewritten Query Q'

Receive user query and query type;
Resolve conversational references;
Expand abbreviations and acronyms;
Correct spelling and normalize terminology;
Generate semantically enriched query Q';
Forward rewritten query to the **Query Decomposition Agent**[A3];
return Q'

Algorithm 4: Query Decomposition Agent

Input: Rewritten Query Q'
Output: Subqueries $\{q_1, q_2, \ldots, q_n\}$

Receive rewritten query;
Analyze query complexity;
if *Query is complex* **then**
    Decompose into ordered subqueries;
else
    Treat the entire query as a single subquery;
Preserve dependency among subqueries;
Forward subqueries to the **Vector Retrieval Agent**[A4];
return $\{q_1, q_2, \ldots, q_n\}$

Algorithm 5: Vector Retrieval Agent

Input: Subquery q_i, Vector Database VDB
Output: Relevant Chunks C

Receive subquery q_i;
Generate query embedding;
Perform top-k similarity search in the vector database;
Retrieve relevant document chunks and entities;
Rank retrieved results according to similarity score;
Forward retrieved evidence to the **Graph Traversal Agent**[A5];
return C

Algorithm 6: Graph Traversal Agent

Input: Relevant Chunks C, Knowledge Graph KG
Output: Graph Evidence E

Receive retrieved entities and document chunks;
Locate corresponding graph nodes;
Perform multi-hop graph traversal;
Collect neighboring entities and relationships;
Construct reasoning paths;
Forward graph evidence to the **Community Retrieval Agent**[A6];
return E

Algorithm 7: Community Retrieval Agent

Input: Graph Evidence E, Knowledge Graph KG
Output: Community Evidence CE

Receive graph evidence;
Identify communities associated with retrieved entities;
Retrieve community summary nodes;
Extract community-level knowledge;
Rank community summaries according to relevance;
Forward community evidence to the **Evidence Aggregation Agent**[A7];
return CE

Algorithm 8: Evidence Aggregation Agent

Input: Vector Evidence V, Graph Evidence E, Community Evidence CE
Output: Aggregated Evidence AE

Receive evidence from all retrieval agents;
Merge retrieved evidence;
Remove duplicate entities and document chunks;
Rank evidence according to relevance and confidence;
Construct a unified evidence set;
Forward aggregated evidence to the **Context Construction Agent**[A8];
return AE

Algorithm 9: Context Construction Agent

Input: Aggregated Evidence AE
Output: Retrieval Context RC

Receive aggregated evidence;
Integrate graph reasoning paths;
Integrate retrieved document chunks;
Attach provenance metadata and citations;
Construct optimized LLM context;
Forward retrieval context to the **Answer Generation Agent**[A9];
return RC

Input: Retrieval Context RC
Output: Generated Answer A

Receive retrieval context;
Prompt the Large Language Model;
Generate grounded natural language answer;
Maintain consistency with retrieved evidence;
Forward generated answer to the **Citation Mapping Agent**[A10];
return A

Algorithm 11: Citation Mapping Agent

Input: Generated Answer A, Retrieval Context RC
Output: Grounded Answer GA

Receive generated answer;
Identify supporting graph nodes;
Map document chunks to generated statements;
Attach provenance metadata and citations;
Generate grounded answer with references;
Forward grounded answer to the **Faithfulness Validation Agent**[A11];
return GA

Algorithm 12: Faithfulness Validation Agent

Input: Grounded Answer GA, Retrieval Context RC
Output: Validation Status VS

Receive grounded answer and retrieval context;
Compare generated statements with supporting evidence;
Measure factual grounding and completeness;
Detect unsupported or hallucinated claims;
Compute faithfulness score;
if *Faithfulness Score* $\geq \tau$ **then**
    Mark answer as Valid;
else
    Mark answer as Invalid;
Forward validation status to the **Self-Critique Agent**[A12];
return VS

Algorithm 13: Self-Critique Agent

Input: Grounded Answer GA, Validation Status VS
Output: Verified Answer VA

Receive grounded answer and validation status;
Perform an independent reasoning pass;
Identify logical inconsistencies;
Verify supporting evidence for each claim;
Suggest corrections when inconsistencies are detected;
Forward verified answer to the **Feedback Agent**[A13];
return VA

Algorithm 14: Feedback Agent

Input: Verified Answer VA, Knowledge Graph KG
Output: Updated Knowledge Graph KG'

Receive verified answer;
Identify incorrect or outdated graph knowledge;
Update confidence scores of graph entities and relationships;
Flag inconsistent graph nodes for refinement;
Record retrieval feedback for future optimization;
Forward updated graph to the **Retry Controller Agent**[A14];
return KG'

Algorithm 15: Retry Controller Agent

Input: Validation Status VS, Knowledge Graph KG', Vector Database VDB
Output: Final Answer A

Receive validation status;
if *Validation Successful* **then**
    Return validated answer;
else
    Modify retrieval parameters;
    Restart retrieval workflow from the **Vector Retrieval Agent**[A4];
    Repeat retrieval until retry limit is reached;
Forward successful retrieval to the **Cache Manager Agent**[A15];
return A

Algorithm 16: Cache Manager Agent

Input: User Query Q, Retrieved Evidence AE, Final Answer A
Output: Updated Cache

Receive query, retrieved evidence, and final answer;
Generate cache key for the query;
Store query embedding;
Store retrieved graph paths and evidence;
Store generated answer with citations;
Update cache index for future retrieval;
Return updated cache;

Table 1: Agents in the Proposed Agentic Graph Retrieval Framework

Agent ID	**Agent Name**	**Responsibility**
A1	Query Classification Agent	Classifies the user query as Local, Global, Hybrid, or Simple Lookup to determine the appropriate retrieval strategy.
A2	Query Rewriting Agent	Normalizes the user query by resolving conversational references, expanding acronyms, correcting ambiguities, and improving semantic representation.
A3	Query Decomposition Agent	Decomposes complex user queries into multiple ordered subqueries for multi-step retrieval and reasoning.
A4	Vector Retrieval Agent	Retrieves the top-k semantically relevant document chunks and entities from the vector database using similarity search.
A5	Graph Traversal Agent	Performs multi-hop traversal over the knowledge graph to discover related entities, relationships, and reasoning paths.
A6	Community Retrieval Agent	Retrieves community summary nodes and graph clusters to efficiently answer global and thematic queries.
A7	Evidence Aggregation Agent	Aggregates, ranks, and removes duplicate evidence collected from vector retrieval and graph traversal.
A8	Context Construction Agent	Constructs the final retrieval context by integrating graph paths, retrieved chunks, provenance, and metadata for LLM reasoning.
A9	Answer Generation Agent	Generates a grounded natural language response using the constructed retrieval context.
A10	Citation Mapping Agent	Maps every generated statement to supporting graph nodes, document chunks, and provenance metadata.
A11	Faithfulness Validation Agent	Evaluates the generated answer for factual grounding, completeness, consistency, and hallucination risk.
A12	Self-Critique Agent	Performs an independent reasoning pass to verify the generated answer and identify inconsistencies or unsupported claims.
A13	Feedback Agent	Updates graph confidence scores, flags inconsistent knowledge, and records validation feedback for future graph refinement.
A14	Retry Controller Agent	Triggers an additional retrieval and reasoning cycle when validation fails, using refined retrieval parameters until a predefined retry limit is reached.
A15	Cache Manager Agent	Caches query embeddings, retrieved evidence, reasoning paths, and generated responses to improve retrieval efficiency and reduce redundant computation.